Code559

Front-End Developer Course

MODULE 10

Forms and Validation with JavaScript

Catch the form before it leaves, check what the visitor typed, and say clearly what to fix.

Free lesson: 559solutions.com/code559/module-10-forms-validation/

In this packet

1. **The lesson**
2. **Project: A Contact form that checks itself, and a rating card that answers**
3. **Finishing work in Git**
4. **Troubleshooting: six common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn to catch a form before it sends, check what was typed, show accessible error messages, and confirm success. It's a kindness to your visitors: a clear message in the right place saves them from guessing. You'll also learn why these checks are about the visitor's experience, not security.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 4
- Core concepts 5
  - 1. How do you catch a form before it leaves? 5
  - 2. How do you read what the visitor typed, and why trim it? 5
  - 3. What does the browser already check for you? 7
  - 4. When do you write your own checks? 8
  - 5. How do you show an error that everyone can perceive? 8
  - 6. Where should the cursor go when something is wrong? 9
  - 7. How do you confirm success and clear the form? 10
  - 8. How do you show what state the form is in? 11
  - 9. Why is client-side validation not security? 13
- Exercise 14
- Common misconception 21
- Summary 21
- Check your understanding 22
- Before next module 22
- Go further 22
- What to keep 22

Part 2: Project: A Contact form that checks itself, and a rating card that answers 24

Part 3: Finishing work in Git 37

Part 4: Troubleshooting: six common problems 39

Part 5: Self-assessment 42

Part 6: Answers 44

About 559 Solutions 52

© 2026 559 Solutions LLC

PART 1

The lesson

Catch the form before it leaves, check what the visitor typed, and say clearly what to fix.

Time: about 2 to 2½ hours (estimate)

In this module you will learn:

- How to handle a form with `addEventListener("submit", …)` and stop the page from reloading with `event.preventDefault()`
- How to read what a visitor typed with `.value`
- Why `.trim()` matters, and how to check for an empty value
- What the browser checks for you: `required`, `type="email"`, `minlength`, `pattern`
- When to add your own JavaScript checks, and what `novalidate` does
- How to show an error message that a screen reader also hears, with `aria-describedby`, and why ARIA is used only when native HTML is not enough
- How to move focus to the first field with a problem with `.focus()`
- How to confirm success with a status message and clear the form with `.reset()`
- How to show form states (`idle`, `sending`, `sent`, `failed`) as classes with `classList`
- Why client-side validation is user experience, not security, and what server-side validation means

---

Why this matters

In Module 4 you built forms in HTML, and the browser already refused an empty `required` field. In Module 9 you learned to find elements and react to a click. A form needs both, plus one more thing: when the visitor presses **Send**, the browser's default is to send the form and load a new page. If you want to check the answers and show a message on the same page, your JavaScript has to step in first.

Good form messages are a kindness. "Please enter your email address," right under the email box, with the cursor already in that box, saves the visitor from guessing. Bad messages, or messages only some visitors can perceive, make people give up.

This module stays on your own computer. Nothing is sent anywhere. A one-second timer stands in for a server so you can practice the states a real form goes through. The last concept explains why the checks you write here protect nobody, and why that is fine.

Before you start

1. Finish Module 9. You should be able to find an element with `querySelector()`, change it with `textContent` and `classList`, and connect a function with `addEventListener()`.

© 2026 559 Solutions LLC

2. In your course folder (for example `Documents\code559`), create a throwaway folder called `practice-10`. Nothing in it is published. Open it in VS Code with **File → Open Folder...**.
3. Keep Edge or Chrome open. You will open the page as a file, as in Module 9, and use **F12 → Console** to read messages and to test lines. No server is needed: a form on a page opened as a file still shows what it sends in the address bar (Exercise step 9).

> **Checkpoint.** You have an empty `practice-10` folder open in VS Code, and you know how to open a file in Edge or Chrome and open the Console.

Core concepts

1. How do you catch a form before it leaves?

A form fires a `submit` event when the visitor presses a submit button or presses Enter in a one-line field, like a text or email box. In a `<textarea>`, Enter starts a new line instead.

Listen on the **form**, not the button. MDN: "The submit event fires on the `<form>` element itself, and not on any `<button>` or `<input type="submit">` inside it" (MDN, *HTMLFormElement: submit event*). The browser hands your function an **event object**, usually named `event`. Its method `preventDefault()` "tells the user agent that the event is being explicitly handled, so its default action... should not be taken" (MDN, *Event: preventDefault() method*). For a form, the default action is sending the data and loading a page. So the first line of every submit handler in this course is the same:

```
function handleSubmit(event) {
  event.preventDefault();
  // now check the fields
}

form.addEventListener("submit", handleSubmit);
```

Notice the pattern from Module 9: `handleSubmit` with no parentheses. The browser calls it, and it passes in `event`.

You will see what happens without this line, for real, in the Exercise break-it step.

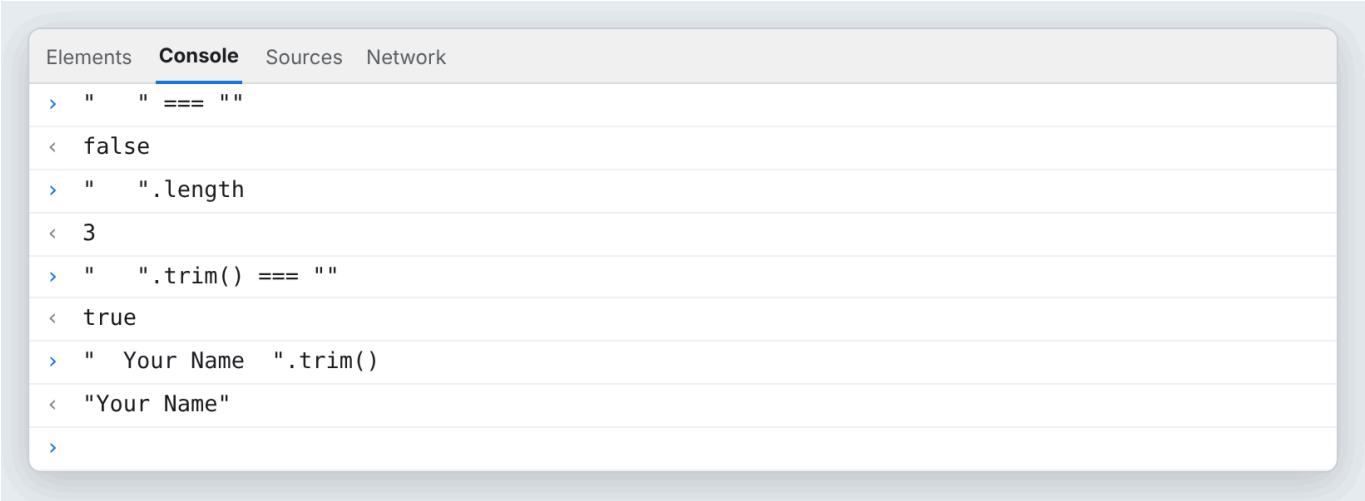
Answer the standing question: **what causes this function to run?** Name two different things a visitor can do that fire `submit`.

2. How do you read what the visitor typed, and why trim it?

Every input and textarea has a `.value`, the text in it right now, as a string.

© 2026 559 Solutions LLC

`nameInput.value` gives whatever is in the box, spaces included. A visitor who presses the space bar three times has typed `" "`, which is **not** the empty string `""`. So a check like `if (nameInput.value === "")` lets three spaces through. `.trim()` "removes whitespace from both ends of this string and returns a new string, without modifying the original string" (MDN, *String.prototype.trim()*). Trim first, then check: `const name = nameInput.value.trim();` and `if (name === "")`. Note the parentheses: `.value` is a property you read, `.trim()` is a method you call, the same difference you met with `.length` and `.push()` in Module 8.



Three spaces have a length of 3, so they are not empty. After `.trim()`, they are. (real output, rendered; evaluated in Chromium 141)

One detail, checked in Chromium: an `<input type="email">` already strips spaces from both ends of its value (`" you@example.com "` came back as `"you@example.com"`), but `type="text"` and `<textarea>` do not (`" Your Name "` came back unchanged). Trim every field anyway. It costs nothing and you never have to remember which type does what.

Predict: what is `" hi ".trim().length`?

Checkboxes and radio buttons hold their answer in a different way. A checkbox has `.checked`, which is `true` or `false`; the Exercise reads `pretendFail.checked`. For a group of radio buttons, ask for the one that is ticked: `document.querySelector('input[name="rating"]:checked')`. The outer single quotes let the double quotes around `rating` stay inside the text. `:checked` is the CSS pseudo-class for a ticked radio button, checkbox or `<option>` (MDN, *:checked*). You get the chosen radio, or `null` if nothing is chosen yet. Its `.value` is the text in its `value` attribute, as a string. Checked in Chromium on a group of five radios named `rating`: with nothing chosen the result was `null`, and after a click on 4, `.value` gave `"4"`. Check for `null` before you read `.value`; `null.value` gives the TypeError you met in Module 9.

© 2026 559 Solutions LLC

3. What does the browser already check for you?

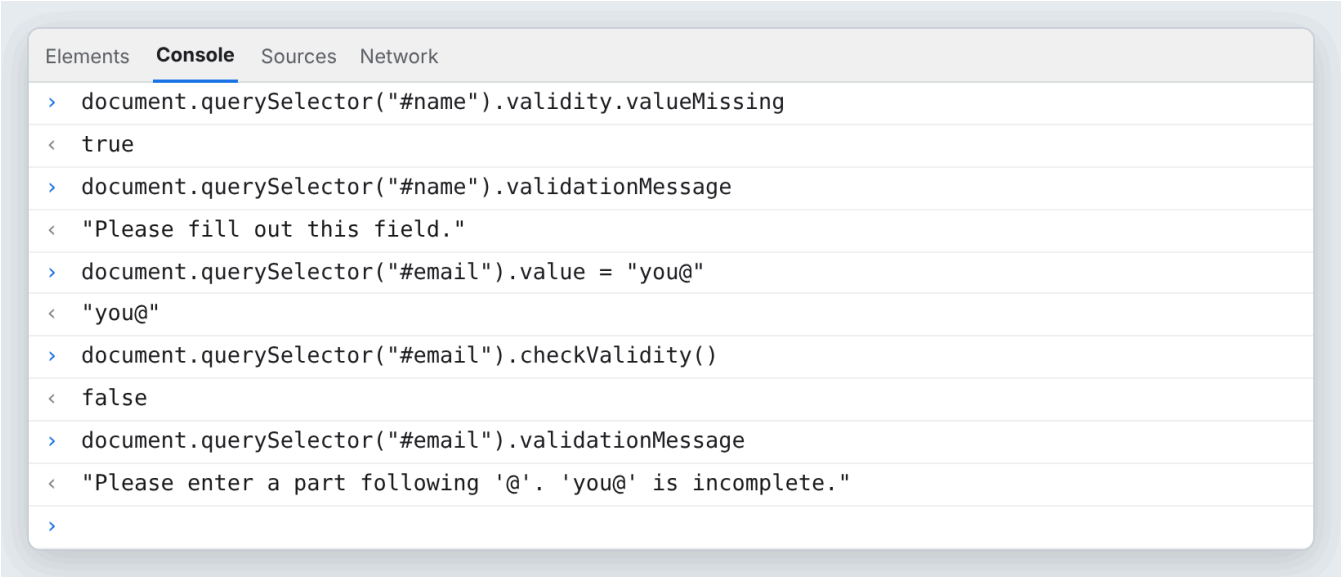
HTML has validation built in. You used `required` in Module 4.

MDN lists the attributes: `required` ("needs to be filled in before the form can be submitted"), `minlength` and `maxlength` (length of text), `min` and `max` (numbers), `type` ("a number, an email address, or some other specific preset type"), and `pattern` ("a regular expression that defines a pattern the entered data needs to follow") (MDN, *Client-side form validation*). Each attribute checks only its own rule. `minlength` does not refuse an empty box: "If an input is not `required`, an empty string can be submitted even if `minlength` is set" (MDN, *minlength HTML attribute*). Checked in Chromium: a textarea with only `minlength="10"` passed `checkValidity()` while empty and failed after typing `hi`. So a field that must be filled in and be long enough needs both `required` and `minlength`. With these on a form and no JavaScript, pressing Send on an empty required field shows the browser's own message bubble and **stops the submit**. Checked in Chromium: with `novalidate` removed, a click on Send fired the `submit` event 0 times and moved focus to the empty Name field. You can read the browser's verdict yourself from the Console.

Typed into the Console for the picture below, in this order. You can type them yourself on the Exercise page once it is open (step 4):

```
document.querySelector("#name").validity.valueMissing
document.querySelector("#name").validationMessage
document.querySelector("#email").value = "you@"
document.querySelector("#email").checkValidity()
document.querySelector("#email").validationMessage
```


© 2026 559 Solutions LLC



`checkValidity()` returns `true` or `false` using the HTML rules. `validationMessage` is the text the browser would show in its bubble; the exact wording differs between browsers and languages. (real output, rendered; evaluated in Chromium 141)

Which attribute would you add to make a "Message" textarea refuse anything under 10 characters?

4. When do you write your own checks?

The built-in bubbles work, but you cannot style them, and they disappear after a moment.

When you want your own messages on the page, you add `novalidate` to the `<form>`. MDN: it "stops the form from showing its own error message bubbles, and allows us to instead display the custom error messages in the DOM," and it "doesn't disable support for the constraint validation API" (MDN, *Client-side form validation*). So you keep the attributes (they still describe the rules, and `checkValidity()` still uses them) and your `submit` handler does the checking. In the Exercise, JavaScript checks three things: the trimmed name is not empty; the email is not empty, and `emailInput.checkValidity()` is `true` (the browser's own `type="email"` rule); the trimmed message is at least 10 characters long. Use built-in checks where they fit, and your own JavaScript where you need a rule HTML cannot say, like "not just spaces."

See the image under Concept 6.

With `novalidate` on, what happens if your `submit` handler does no checking at all?

5. How do you show an error that everyone can perceive?

An error message must be seen **and** heard. A red border alone tells a screen-reader user nothing, and color alone fails visitors who do not see red.

© 2026 559 Solutions LLC

Put an empty element for the message right after each field, and tie it to the field with `aria-describedby`:

```
<input id="email" name="email" type="email" required aria-describedby="email-error">
<p id="email-error" class="error"></p>
```

MDN: `aria-describedby` "lists the ids of the elements that describe the object" (MDN, *ARIA: aria-describedby attribute*). When the field gets focus, a screen reader reads its label, then its description, which is your message. JavaScript fills the message with `textContent` and adds a class for the red border; it clears both when the field is fixed. Checked in Chromium's accessibility tree: after the email error appeared, the email field was a "textbox" named "Email" with the description "That email address looks incomplete. Check the @ and the part after it."

This is the only ARIA in the form, on purpose. MDN quotes the first rule of ARIA: "If you can use a native HTML element or attribute with the semantics and behavior you require already built in... then do so" (MDN, *ARIA*). The `<label for>` already names each field. `required` already says it is required. `aria-describedby` is here because HTML has no native way to say "this paragraph is the error for that input." **ARIA only when native HTML is not enough.**

The image under Concept 6 shows two filled message elements.

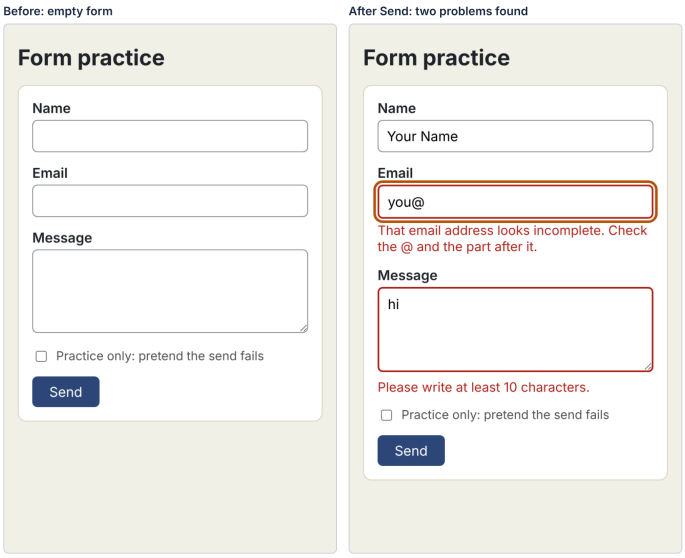
Why is the message element in the HTML from the start, empty, instead of created by JavaScript when the error happens? (Hint: `aria-describedby` points to an id. What has to exist for it to point at?)

6. Where should the cursor go when something is wrong?

After a failed submit, the visitor's next question is "what do I fix?" Move them there.

© 2026 559 Solutions LLC

Every element that can take focus has a `.focus()` method. The checking function in the Exercise, `findFirstProblem()`, checks every field, shows every message, and **returns the first field with a problem, or `null`** if there is none, the same return-`null` pattern from Module 7. The submit handler then calls `firstProblem.focus()`. Focus lands in the field, a keyboard user can type the fix at once, and a screen reader reads the label and the error together.



Name was fine, so the first problem is Email, and that is where focus went (the dark orange ring around its red border). Both problems are listed at once, so the visitor fixes everything in one pass. Labels added above each render. (real: rendered in Chromium)

If all three fields had problems, which field would receive focus? Why that one?

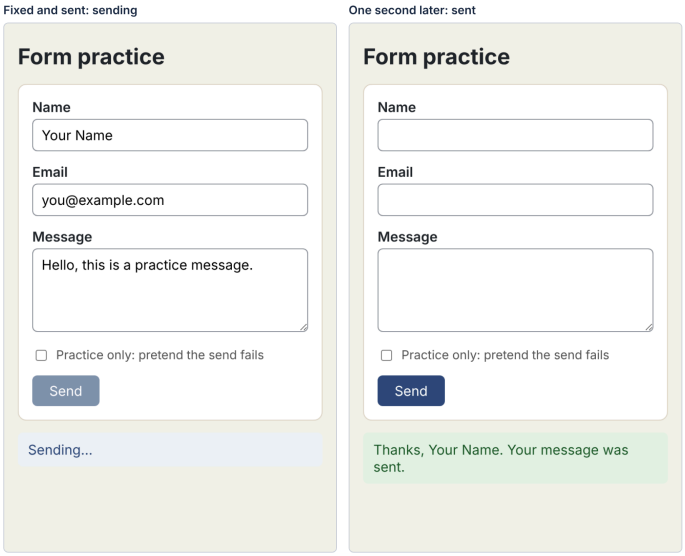
7. How do you confirm success and clear the form?

When everything passes, say so, clearly, and get the form ready for the next message.

The confirmation goes into a status element that is in the HTML from the start: `<p id="status" role="status"></p>`. MDN's live-regions guide describes `role="status"` as "an area of the screen that provides an updated status of some kind"; it is a polite live region, which the guide says a screen reader speaks "whenever the user is idle." The same guide warns: "Establish the live region before updating its content... The most reliable way to ensure that live regions are registered is to include

© 2026 559 Solutions LLC

them in the initial markup" (MDN, *ARIA live regions*). Then `form.reset()`, which "restores a form element's default values" (MDN, *HTMLFormElement: reset() method*). Read anything you want to show (like the name) **before** you reset, or it will be gone.



The fixed form went through `sending` and then `sent`. The name in the thank-you line was read before `form.reset()` emptied the fields. Labels added above each render. (real: rendered in Chromium)

Put these in order: `form.reset()`, read the name, show the thank-you message.

8. How do you show what state the form is in?

A real form is not just "filled" or "empty." It is waiting, or busy, or done, or it failed.

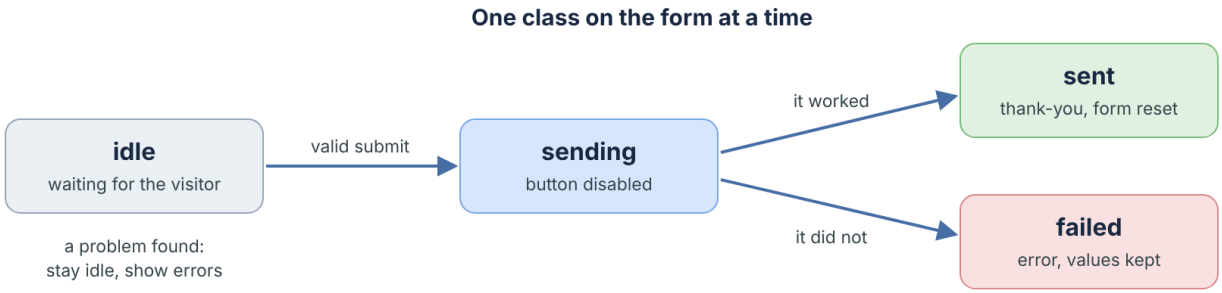
Give the form exactly one state class at a time and let CSS draw each state. One small function does it, using two `classList` methods from Module 9:

```
function setState(stateName) {
  form.classList.remove("idle", "sending", "sent", "failed");
  form.classList.add(stateName);
}
```

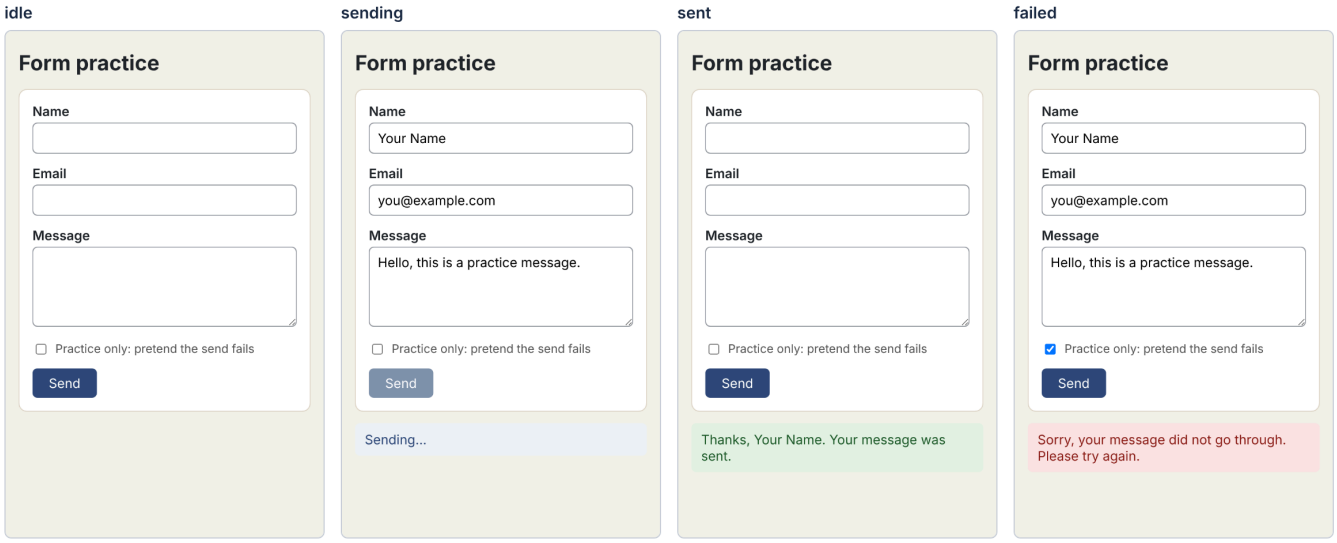
`remove()` accepts several class names at once; classes that are not there are ignored. While sending, the button is also disabled with `sendButton.disabled = true`, so a second click cannot send twice. In the Exercise, `setTimeout(finishSending, 1000)` waits one second and then calls `finishSending`, a stand-in for a server's answer. (Same rule as listeners: you pass `finishSending`, not

`finishSending()`.) A practice checkbox lets you choose whether the pretend send fails, so you can see every state. In a later, optional module (not yet published), a real server replaces the timer. **Nothing in this module leaves your computer.**

One side effect of disabling Send, checked in Chromium: if focus is on the button when it becomes disabled (after a click, or Tab and Enter), focus drops to the page itself, and `document.activeElement` is `<body>`. Focus stays there after the button comes back, so a keyboard user presses Shift+Tab to get back into the form. Pressing Enter in the Name or Email box does not have this problem: focus stays in that box.



Four states, one class at a time. When the send fails, the values stay, so the visitor does not retype a long message.



The same page in each of the four states, driven only by the class on the `<form>` and the text in `#status`. Labels added above each render. (real: rendered in Chromium)

Which CSS selector in the Exercise colors the status message red? Find it before you type the CSS.

9. Why is client-side validation not security?

Everything in this module runs in the visitor's browser. The visitor controls that browser.

MDN is direct: client-side validation "should not be considered an exhaustive security measure," because it "is too easy to bypass," and "Your apps should always perform validation, including security checks, on any form-submitted data on the server-side as well" (MDN, *Client-side form validation*). Anyone can open DevTools and delete your `required` attributes, switch off JavaScript, or skip your page entirely and send data straight to the server. So the rule for this course is: **client-side validation is user experience, not security.** It helps honest visitors fix mistakes quickly. **Server-side validation** means the program that receives the data checks it again, every time, before trusting it. Writing that program is outside this course's core; you only need to know it must exist.

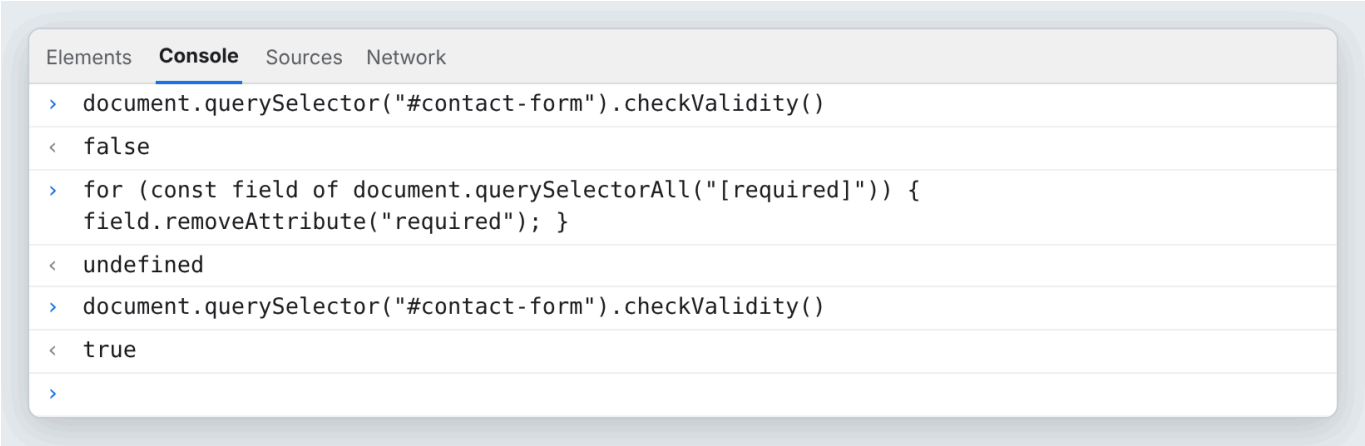


Typed into the Console for the picture below, in this order. You can type them yourself on the Exercise page once it is open (step 4):

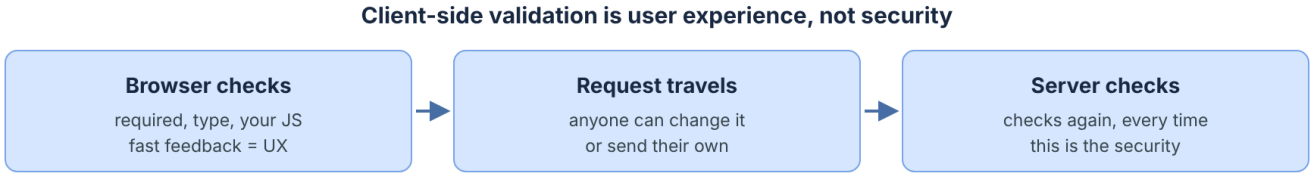
```
document.querySelector("#contact-form").checkValidity()
for (const field of document.querySelectorAll("[required]")) {
field.removeAttribute("required"); }
document.querySelector("#contact-form").checkValidity()
```

Press **F5** afterwards to get the `required` attributes back.

© 2026 559 Solutions LLC



One line in the Console, using Module 9's `querySelectorAll()` and `for...of`, switched the browser's rules off. The empty form now counts as valid. (real output, rendered; evaluated in Chromium 141)



Your checks live in the first box. Only the last box protects anything.

A friend says, "My form has `required` on every field, so no one can send me an empty message." What one sentence do you say back?

Exercise

Your hands, about 75 minutes (estimate). Type it, do not paste it. Work in `practice-10`. Nothing here is published, and nothing is sent anywhere.

1. **Create the HTML.** Make `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form practice</title>
  <link rel="stylesheet" href="styles.css">
  <script src="script.js" defer></script>
</head>
<body>
  <main>
```


© 2026 559 Solutions LLC

```
    <h1>Form practice</h1>
    <form id="contact-form" class="idle" novalidate>
      <div class="field">
        <label for="name">Name</label>
        <input id="name" name="name" type="text" required aria-
describedby="name-error">
        <p id="name-error" class="error"></p>
      </div>
      <div class="field">
        <label for="email">Email</label>
        <input id="email" name="email" type="email" required aria-
describedby="email-error">
        <p id="email-error" class="error"></p>
      </div>
      <div class="field">
        <label for="message">Message</label>
        <textarea id="message" name="message" rows="4" required minlength="10"
aria-describedby="message-error"></textarea>
        <p id="message-error" class="error"></p>
      </div>
      <div class="field check">
        <input id="pretend-fail" type="checkbox">
        <label for="pretend-fail">Practice only: pretend the send fails</label>
      </div>
      <button id="send-btn" type="submit">Send</button>
    </form>
    <p id="status" role="status"></p>
  </main>
</body>
</html>
```

Notice: `novalidate` on the form, `class="idle"` to start, each field's `aria-describedby` naming the empty `<p>` under it, and the `#status` paragraph outside the form, right after it.

2. **Add the CSS.** Make `styles.css`. The last block is the state styles. `.sent + #status` means "the `#status` element right after a form with the class `sent`." `:empty` matches an element with nothing inside it, not even a space (MDN, *:empty*). So `.error:empty` hides an error line until it has text. `#status:empty` only takes away the padding and margin: the empty status line takes no space, but it stays in the page, ready for screen readers (Concept 7). Checked in Chromium's accessibility tree: the empty `#status` is there at load, as a polite `status` region.

© 2026 559 Solutions LLC

Why `input.invalid` and not just `.invalid`? Module 2's specificity: `input[type="text"]` scores 0-1-1, because an attribute selector counts in the class column, so it beats `.invalid` (0-1-0); `input.invalid` also scores 0-1-1 and wins the tie because it comes later in the file.

```
body {
  margin: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  background: #f4f1ea;
  color: #1f2328;
}
main { max-width: 520px; margin: 0 auto; padding: 24px 16px; }
h1 { font-size: 1.6rem; margin: 0 0 16px; }
form {
  background: #fff;
  border: 1px solid #ddd5c4;
  border-radius: 10px;
  padding: 16px;
}
.field { margin-bottom: 14px; }
label { display: block; font-weight: 600; margin-bottom: 4px; }
.check { display: flex; gap: 8px; align-items: center; }
.check label { font-weight: 400; margin: 0; font-size: 0.9rem; color: #555; }
input[type="text"], input[type="email"], textarea {
  box-sizing: border-box;
  width: 100%;
  font: inherit;
  padding: 8px 10px;
  border: 1px solid #8a8f98;
  border-radius: 6px;
}
input:focus-visible, textarea:focus-visible, button:focus-visible {
  outline: 3px solid #b45309;
  outline-offset: 2px;
}
input.invalid, textarea.invalid { border: 2px solid #b3261e; }
.error { color: #b3261e; margin: 4px 0 0; font-size: 0.95rem; }
.error:empty { display: none; }
button {
  font: inherit;
  padding: 8px 20px;
  border: 0;
  border-radius: 6px;
  background: #2d4a7a;
  color: #fff;
  cursor: pointer;
}
```


© 2026 559 Solutions LLC

```
/* Form states */
.sending button { opacity: 0.6; cursor: wait; }
#status { margin: 14px 0 0; padding: 10px 12px; border-radius: 6px; }
#status:empty { padding: 0; margin: 0; }
.sent + #status { background: #e3f4e5; color: #1e5b2a; }
.failed + #status { background: #fbe4e2; color: #8c1d18; }
.sending + #status { background: #eef1f6; color: #2d4a7a; }
```

3. **Type the script.** Make `script.js`. Type the three block comments first. Then fill block 1, then the small functions in block 2, then `findFirstProblem`, then `finishSending`, then `handleSubmit`, then block 3.

```
// 1. Find elements
const form = document.querySelector("#contact-form");
const nameInput = document.querySelector("#name");
const emailInput = document.querySelector("#email");
const messageInput = document.querySelector("#message");
const nameError = document.querySelector("#name-error");
const emailError = document.querySelector("#email-error");
const messageError = document.querySelector("#message-error");
const pretendFail = document.querySelector("#pretend-fail");
const sendButton = document.querySelector("#send-btn");
const statusMessage = document.querySelector("#status");

// 2. Define behavior
function setState(stateName) {
  form.classList.remove("idle", "sending", "sent", "failed");
  form.classList.add(stateName);
}

function showError(input, errorElement, text) {
  errorElement.textContent = text;
  input.classList.add("invalid");
}

function clearError(input, errorElement) {
  errorElement.textContent = "";
  input.classList.remove("invalid");
}

// Checks every field. Returns the first field with a problem, or null.
function findFirstProblem() {
  let firstProblem = null;
  clearError(nameInput, nameError);
  clearError(emailInput, emailError);
  clearError(messageInput, messageError);
```


© 2026 559 Solutions LLC

```
  const name = nameInput.value.trim();
  if (name === "") {
    showError(nameInput, nameError, "Please enter your name.");
    firstProblem = nameInput;
  }

  const email = emailInput.value.trim();
  if (email === "") {
    showError(emailInput, emailError, "Please enter your email address.");
    if (firstProblem === null) { firstProblem = emailInput; }
  } else if (!emailInput.checkValidity()) {
    showError(emailInput, emailError, "That email address looks incomplete.
Check the @ and the part after it.");
    if (firstProblem === null) { firstProblem = emailInput; }
  }

  const message = messageInput.value.trim();
  if (message.length < 10) {
    showError(messageInput, messageError, "Please write at least 10
characters.");
    if (firstProblem === null) { firstProblem = messageInput; }
  }

  return firstProblem;
}

function finishSending() {
  if (pretendFail.checked) {
    setState("failed");
    statusMessage.textContent = "Sorry, your message did not go through. Please
try again.";
  } else {
    setState("sent");
    statusMessage.textContent = "Thanks, " + nameInput.value.trim() + ". Your
message was sent.";
    form.reset();
  }
  sendButton.disabled = false;
}

function handleSubmit(event) {
  event.preventDefault();
  statusMessage.textContent = "";
  const firstProblem = findFirstProblem();
  if (firstProblem !== null) {
    setState("idle");
```


© 2026 559 Solutions LLC

```
    firstProblem.focus();
    return;
  }
  setState("sending");
  statusMessage.textContent = "Sending...";
  sendButton.disabled = true;
  // Stand-in for a real server: wait one second, then finish.
  setTimeout(finishSending, 1000);
}

// 3. Connect events
form.addEventListener("submit", handleSubmit);
```

4. **Predict, then test empty.** Open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. Predict what happens when you press **Send** with every field empty: how many messages, and where does the cursor go? Then press Send.

5. **Test the tricky values.** Type three spaces in Name, `you@` in Email and `hi` in Message. Press Send. Compare with the right half of the image under Concept 6 (that one had a real name, so Name passed).

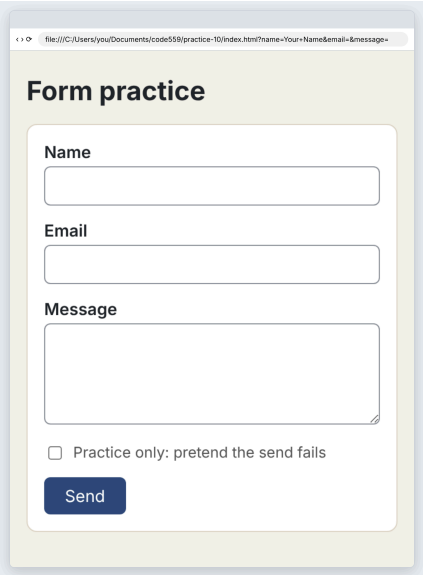
6. **Fix and send.** Fix all three fields and press Send. Watch for `Sending...` and the faded button, then the green thank-you one second later. The fields should be empty.

7. **Test the failure.** Fill the form again, tick **Practice only: pretend the send fails**, and press Send. Your values should still be in the fields, above the red message.

8. **Keyboard only.** Reload. Use only Tab, typing and Enter to fill the form with one mistake and submit it. Pressing Enter in the Name or Email box submits the form too. In the Message box, Enter starts a new line, so Tab to **Send** and press Enter there. Does focus land on the mistake?

9. **Break it on purpose.** In `handleSubmit`, comment out the first line: `// event.preventDefault();`. Save and press **F5**. Type a name, leave the rest empty, and press Send. Watch the address bar.

© 2026 559 Solutions LLC

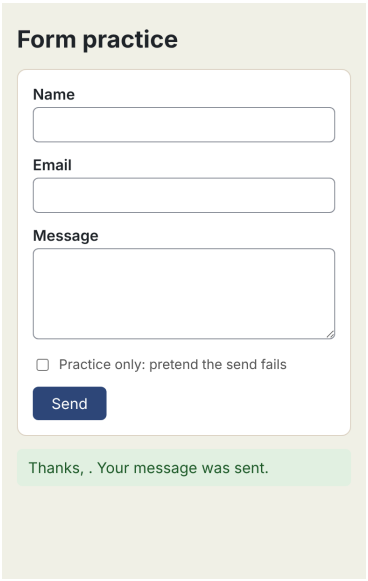


Your handler ran and even wrote the error messages, but then the browser did its default: it sent the form by adding `name=value` pairs to the address and loaded the page again. The new page starts fresh, so the name you typed and every message are gone. The `?name=Your+Name&email=&message=` part is real, captured from Chromium after the click on the page opened as a file; the frame around it is drawn. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

The clue is the address bar, not the Console: nothing went red. Remove the `//` and save. Then delete the `?…` part of the address, press Enter to load the page again, and test again.

10. **Optional second break.** In `findFirstProblem`, change `const name = nameInput.value.trim();` to `const name = nameInput.value;`. Save and press **F5**. Type three spaces as the name, fill the rest correctly, and press Send.

© 2026 559 Solutions LLC



Three spaces are not `""`, so the check passed and the form "sent" with no name. The thank-you line still trims, so the visitor sees "Thanks, ." (real: rendered in Chromium)

Put `.trim()` back.

Common misconception

> *"If my JavaScript checks every field, the data that reaches me will always be clean."*

What would you say? Hint: look again at the image under Concept 9. Who controls the browser your checks run in?

Summary

A form fires `submit` on the form itself, and `event.preventDefault()` stops the browser from sending it and reloading the page. `.value` gives what the visitor typed as a string, and `.trim()` removes spaces at both ends so that three spaces count as empty. HTML's `required`, `type`, `minlength` and `pattern` check input with no JavaScript and block the submit. `novalidate` turns off the browser's bubbles so your own messages can appear on the page, while `checkValidity()` still uses the HTML rules. Each error message sits in an element tied to its field with `aria-describedby`, the one place native HTML is not enough. `.focus()` moves the visitor to the first field with a problem. A `role="status"` element in the initial HTML announces the confirmation, and `.reset()` clears the form after you have read what you need. One state class at a time, set with `classList`, lets CSS show `idle`, `sending`, `sent` and `failed`. Client-side validation is user experience, not security; the server must always check again.

© 2026 559 Solutions LLC

Check your understanding

1. Your submit handler shows the right errors, but they flash and vanish, and the address bar gains a `?`. What is missing?
2. **What value does this variable hold?** A visitor types two spaces, then `Ana`, then one space into Name. What does `nameInput.value` hold, and what does `nameInput.value.trim()` hold?
3. Why does the Exercise keep the `required` and `minlength` attributes even though `novalidate` is on and JavaScript does the checking?

Before next module

- [ ] Your `practice-10` form shows every problem at once and puts focus on the first one
- [ ] Three spaces in Name are refused
- [ ] You saw all four states: idle, sending, sent, failed
- [ ] You saw the page reload and lose its messages without `preventDefault()`, and fixed it
- [ ] You can say in one sentence why client-side validation is not security
- [ ] If you did this module's packet project in my-site, saved it with the routine: git status → git add . → git commit -m "..." → git push → check the live page. (Your practice folder is not saved to GitHub.)

Go further

- MDN: *Client-side form validation* (Learn web development, Forms), the whole page, including "Validating forms using JavaScript"
- MDN: *HTMLFormElement: submit event* and *Event: preventDefault() method*
- MDN: *String.prototype.trim()*
- MDN: *HTMLFormElement: reset() method*
- MDN: *ARIA: aria-describedby attribute*; *ARIA live regions*; *ARIA* (the "Before using ARIA" section and the first rule of ARIA)
- freeCodeCamp: the free JavaScript Certification (checked 2026-10-11) has practice on this topic.

What to keep

Keep `practice-10` with its finished `script.js`, and write down the five tests from steps 4 to 8 as one line each: what you did, what you expected, what happened. Module 11 turns that list into a test plan. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "Client-side form validation"; MDN "HTMLFormElement: submit event"; MDN "Event: preventDefault() method"; MDN "String.prototype.trim()"; MDN "HTMLFormElement: reset() method"; MDN "ARIA: aria-describedby attribute"; MDN "ARIA live regions"; MDN "ARIA". Checked 2026-10-11: MDN ":checked"; MDN ":empty"; MDN "Handling conflicts" (attribute selectors count in the class column); MDN "HTMLFormElement: submit event" (pressing Enter in a text input); MDN "minlength HTML attribute". Every page render, Console result and accessibility-tree check was captured in Chromium 141.0.7390.37 (via Playwright) on Linux.

© 2026 559 Solutions LLC

PART 2

Project: A Contact form that checks itself, and a rating card that answers

Your hands, about 2½ hours (estimate). In the Exercise you built the whole validation cycle in `practice-10`. Now you put it on your real site, twice. The **Contact page** gets the full cycle: messages in your own voice, focus on the first problem, a confirmation that names the visitor, four form states, and a keyboard-only test you write down and run. The **rating card** from Module 4 gets its JavaScript at last: it reads the chosen rating and swaps the form for the thank-you section. You use only what Modules 4 to 10 taught:

- `<script src="…" defer>` in the `<head>`, one script per page, and the three blocks: find elements, define behavior, connect events (Module 9)
- `novalidate`, `required`, `type="email"`, `minlength`, `aria-describedby` and `role="status"` in the HTML
- a `submit` listener whose first line is `event.preventDefault()`
- `.value`, `.trim()`, `.length`, `=== ""` and `checkValidity()` to check what was typed
- `querySelector('input[name="rating"]:checked')`, and `null` checked with `===` (Core concept 2)
- `textContent`, `classList.add()` and `.remove()`, `hidden`, `.disabled`, `.focus()` and `.reset()`
- `setTimeout(finishSending, 1000)`, the lesson's one-second stand-in for a server
- functions that return a value or `null` (Module 7), and a `let` you change from the Console
- custom properties, `.visually-hidden` (Module 4), and the `:empty` and `+` selectors from the Exercise CSS

This project starts from the Module 9 packet's site: five pages, one shared `styles.css` with the colors and focus rings, `gallery.js` on the Interests page and `faq.js` on the About page. The Rate and Contact pages are as you built them in the Module 4 packet, with no script yet.

The brief

Read the whole brief twice before you type anything.

When you finish, `my-site` holds two new files, `contact.js` and `rate.js`, next to `index.html`. `contact.html`, `rate.html` and `styles.css` have changed. No other file changes. As in Module 9, each page loads only its own script: `contact.html` loads `contact.js`, and `rate.html` loads `rate.js`, each with one `<script src="…" defer>` line under the stylesheet link.

One sentence to keep in mind. These checks help honest visitors fix mistakes quickly. They protect nothing: anyone can switch them off (Core concept 9). Client-side validation is user experience, not security. If you connect this form to a real server one day, that server must check every message

© 2026 559 Solutions LLC

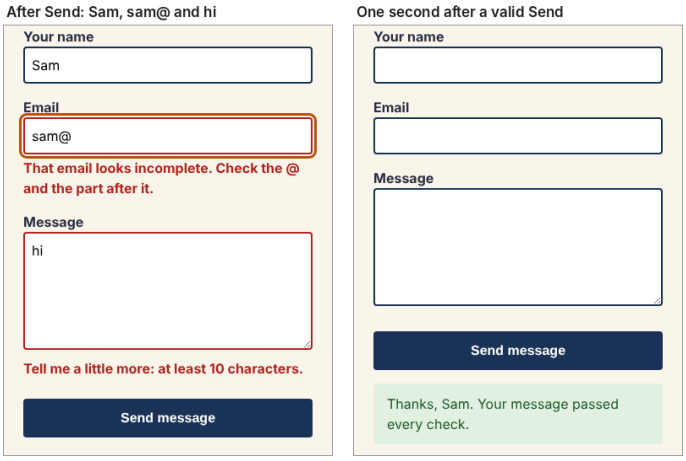
again.

The Contact page HTML. The page keeps its heading, its lead sentence and its three labeled fields. These things are new:

- Under the lead sentence, a `<p class="note">` that says exactly: **This form checks your message, but it is not connected to a server yet. Nothing you type here is sent.** The `note` class is your Module 2 box.
- The form gets `novalidate`, and the class `idle` next to `contact-form`.
- Under each field, an empty `<p class="error">`. The ids are `name-error`, `email-error` and `message-error`. Each field points to its own with `aria-describedby`.
- The Message box also gets `minlength="10"`. The `required` attributes stay (Check your understanding 3).
- Right after the form, an empty `<p id="contact-status" role="status">`.

What the Contact page does. On **Send message**, the script checks three things. The name, trimmed, is not empty. The email is not empty, and the browser's `type="email"` rule accepts it. The message, trimmed, is at least 10 characters long. It shows every problem at once, each under its own field, and puts focus in the first field with a problem. A field with a problem also gets a red border, but the words carry the meaning, not the color.

When everything passes, the form goes through the states from Core concept 8. **Sending:** the button fades and is disabled for one second, and the status line says **One moment…**. **Sent:** the status line thanks the visitor by name, as in **Thanks, Sam. Your message passed every check.**, and the form is empty again. **Failed:** for testing only, typed in the Console. The status line says what went wrong, and the visitor's words stay in the fields. The form always has exactly one of the four classes: `idle`, `sending`, `sent` or `failed`.



The finished Contact page. Left: Name was fine, so focus went to Email, the first field with a problem. Right: the `sent` state, with the name read before the form was cleared. Labels added above each render. (real: rendered in Chromium at 390 px)

© 2026 559 Solutions LLC

Your own voice. The three error messages, the thank-you and the failed message are yours to write. Each error says what to do, not only what is wrong. None of them blames the visitor. And because nothing is sent, no message may say it was sent: the sample says "passed every check," not "was sent."

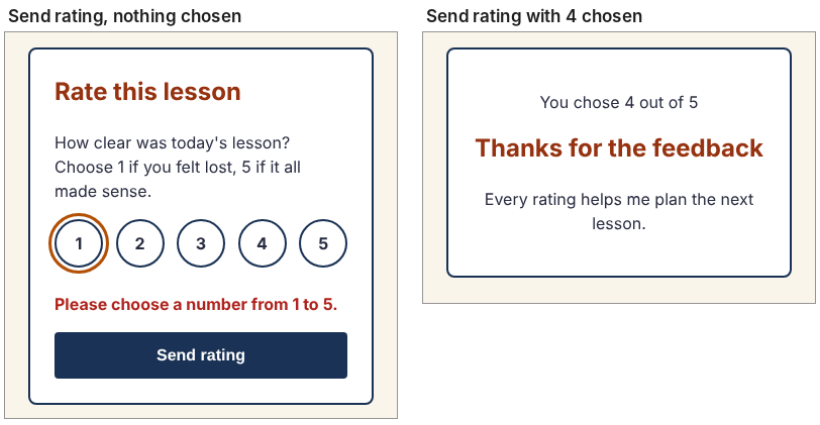
The rating card HTML. The card keeps everything from Module 4. These things are new:

- The rating form gets `novalidate`.
- The fieldset gets `aria-describedby="rating-error"`, and right after the fieldset comes an empty `<p id="rating-error" class="error">`.
- At the end of the card, after the thank-you section, an empty `<p id="rating-status" class="visually-hidden" role="status">`.

What the rating card does. On **Send rating** with nothing chosen, the card says **Please choose a number from 1 to 5.** under the numbers and puts focus on number 1. The dark orange ring shows on 1 only when you used the keyboard: after a mouse click, Chromium moves focus but draws no ring, because `:focus-visible` follows the keyboard.

With a number chosen, the form disappears and the thank-you section appears, with the number in its span: **You chose 4 out of 5**. The button that had focus is now hidden, so focus goes back to the page itself, and the next Tab starts again from the top of the page.

At the same moment, the hidden status line gets one sentence, **Thanks for the feedback. You chose 4 out of 5.** Sighted visitors never see it. It is there because the thank-you section starts hidden, so it cannot be a live region from the start (Core concept 7). The status line can.



The rating card driven by keyboard only. Left: Tab to Send rating, then Enter, with nothing chosen. Right: three presses of the right arrow, Tab, then Enter. Labels added above each render. (real: rendered in Chromium at 390 px)

The colors. One new custom property, `--error: #b3261e`, for the messages and the red border. It is 6.1:1 on your cream page background and 6.5:1 on the white card. The status line uses `--ink` on `--soft` while sending (10.8:1), dark green `#1e5b2a` on pale green `#e3f4e5` when sent (7.1:1), and dark

© 2026 559 Solutions LLC

red `#8c1d18` on pale red `#fbe4e2` when failed (7.5:1). These values were checked with the WCAG contrast formula.

The faded button is disabled for that one second, and WCAG sets no contrast minimum for "an inactive user interface component" (WCAG 2.2, Success Criterion 1.4.3). The focus rings are your Module 4 rings. Nothing scrolls sideways at 390 px.

The ids, classes, file names and the note must match the brief. The error messages, the thank-you and the failed message are yours.

The steps

Type every line. Do not paste. Many lines look like the Exercise: type them again anyway, and notice what changed.

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`.

2. **Write your messages on paper.** Write the five sentences that are yours: an error for an empty name, for an empty email, for an incomplete email, and for a short message, then the thank-you with a place for the name. Read each one as a visitor who just made that mistake. Does it say what to do next? Then write the three block headings from Module 9 for `contact.js`, and under each, what goes there.

3. **The Contact HTML.** Open `contact.html`. In the `<head>`, under the stylesheet link, add:

   ```
   <script src="contact.js" defer></script>
   ```

   Then change the inside of the `<section>`, under the `<h2>`. Your lead sentence stays where it is. Read it once more: now that the page says nothing is sent, it must not promise a reply.

   ```
   <!-- your lead sentence stays here -->
   <p class="note">This form checks your message, but it is not connected to a
   server yet. Nothing you type here is sent.</p>
   <form class="contact-form idle" novalidate>
     <label for="visitor-name">Your name</label>
     <input type="text" id="visitor-name" name="visitor-name" required aria-
   describedby="name-error">
     <p id="name-error" class="error"></p>

     <!-- Continue alone: give the email input aria-describedby="email-error"
          and put an empty p with id="email-error" and class="error" under it.
          Do the same for the textarea with message-error, and give the
          textarea minlength="10". -->
   ```


© 2026 559 Solutions LLC

```
  <button type="submit">Send message</button>
</form>
<p id="contact-status" role="status"></p>
```

Predict before you test: `contact.js` does not exist yet, and the form now has `novalidate`. What happens when you press **Send message** with all three fields empty? Write it down. Then open `contact.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. Press Send message.

The sample page reloaded at once. The address ended in `contact.html?visitor-name=&visitor-email=&message=`, and the Console had one red line, `Failed to load resource: net::ERR_FILE_NOT_FOUND`, with `contact.js` beside it. Nobody checked anything. `novalidate` switched off the browser's checks, and the script that should replace them is missing. Delete the `?…` part of the address and press Enter to load the page again.

4. **The CSS for messages and states.** Open `styles.css`. In `:root`, under `--ring`, add one line:

```
--error: #b3261e;
```

Then, above the `/* Wider screens: 700px and up */` comment, under your FAQ group, add a new group:

```
/* Forms: error messages and states */
.error {
  color: var(--error);
  font-weight: 700;
  margin: 4px 0 0;
}

.error:empty {
  display: none;
}

.contact-form input.invalid,
.contact-form textarea.invalid {
  border-color: var(--error);
}

.sending button {
  opacity: 0.6;
  cursor: wait;
}
```


© 2026 559 Solutions LLC

```
#contact-status {
  max-width: 480px;
  margin: 16px 0 0;
  padding: 12px 16px;
  border-radius: 4px;
}

#contact-status:empty {
  margin: 0;
  padding: 0;
}

.sending + #contact-status {
  color: var(--ink);
  background-color: var(--soft);
}

/* Continue alone: .sent + #contact-status gets color #1e5b2a on
   background-color #e3f4e5. .failed + #contact-status gets color
   #8c1d18 on background-color #fbe4e2. */
```

Why `.contact-form input.invalid`? Your Module 4 rule `.contact-form input` scores 0-1-1 and sets a navy border. The new selector scores 0-2-1, so it wins wherever it matches (Module 2). It changes only `border-color`, so the width stays 2 px.

5. **`contact.js`, block 1.** In the Explorer, create `contact.js` next to `contact.html`. Type:

```
// contact.js: checks the Contact form, shows clear messages, and shows four
states

// 1. Find elements
const form = document.querySelector(".contact-form");
const nameInput = document.querySelector("#visitor-name");
const emailInput = document.querySelector("#visitor-email");
const messageInput = document.querySelector("#message");
const nameError = document.querySelector("#name-error");
// Continue alone: emailError and messageError, then sendButton
// (the selector ".contact-form button") and statusMessage ("#contact-status").

// For testing only: type pretendFail = true in the Console to see the failed
state.
let pretendFail = false;
```

The form has a class, not an id, so the selector starts with a dot. `pretendFail` replaces the practice checkbox from the Exercise. A test switch does not belong on a public page, so it lives in the script, where only you will change it. It is a `let`, because you change it from the Console

© 2026 559 Solutions LLC

in step 10.

Save and press F5. The red line about `contact.js` is gone. Test block 1 in the Console: type `emailError`, then `pretendFail`. The sample showed the empty paragraph `<p id="email-error" class="error"></p>`, then `false`.

6. **Block 2: three small functions.** Under block 1, leave one blank line and type:

```
// 2. Define behavior
function setState(stateName) {
  form.classList.remove("idle", "sending", "sent", "failed");
  form.classList.add(stateName);
}

// Continue alone: showError(input, errorElement, text) puts the text
// in errorElement and adds the class "invalid" to input.
// clearError(input, errorElement) empties errorElement and removes "invalid".
```

Write `showError` and `clearError` from their descriptions, as you wrote helpers in Module 7. Then test all three by hand. Save, press F5, and type `showError(nameInput, nameError, "Test")`. The sample showed **Test** in bold red under the Name box, and the box got a red border. Type `clearError(nameInput, nameError)`: both were gone. Then type `setState("sent")` and `form.className`. The sample gave `'contact-form sent'`: the `idle` class was gone, and `contact-form` stayed.

7. **`findFirstProblem`: your own messages.** Under the small functions, type:

```
// Checks every field. Returns the first field with a problem, or null.
function findFirstProblem() {
  let firstProblem = null;
  clearError(nameInput, nameError);
  clearError(emailInput, emailError);
  clearError(messageInput, messageError);

  const name = nameInput.value.trim();
  if (name === "") {
    showError(nameInput, nameError, "I'd love to know who is writing. Please add
your name.");
    firstProblem = nameInput;
  }

  // Continue alone: the email (empty, or checkValidity() is false)
  // and the message (trimmed length under 10). Use your own messages.
  // Set firstProblem only if it is still null.
```


© 2026 559 Solutions LLC

```
  return firstProblem;
}
```

The shape is the Exercise's: clear every message, check every field, remember only the first problem. Put your own sentences from step 2 in the quotes. The sample's are in Part 6, *Answers*.

Test it by hand before any button uses it. Save, press F5, and type `findFirstProblem()` with all three fields empty. The Console showed the Name box itself, `<input type="text" id="visitor-name" …>`, and all three messages appeared on the page. Fill in `Sam`, `sam@example.com` and `Hello, I like your site.`, then call it again. The sample gave `null`, and every message was gone.

8. **Finish, submit, connect.** Under `findFirstProblem`, type `finishSending` and `handleSubmit`, then block 3:

```
function finishSending() {
  if (pretendFail) {
    // Continue alone: set the state to "failed" and put your
    // failed message in statusMessage. Do not reset the form.
  } else {
    const name = nameInput.value.trim();
    setState("sent");
    statusMessage.textContent = "Thanks, " + name + ". Your message passed every
check.";
    form.reset();
  }
  sendButton.disabled = false;
}

function handleSubmit(event) {
  event.preventDefault();
  statusMessage.textContent = "";
  const firstProblem = findFirstProblem();
  if (firstProblem !== null) {
    setState("idle");
    firstProblem.focus();
    return;
  }
  setState("sending");
  statusMessage.textContent = "One moment...";
  sendButton.disabled = true;
  // Stand-in for a real server: wait one second, then finish.
  setTimeout(finishSending, 1000);
}
```


© 2026 559 Solutions LLC

```
// 3. Connect events
form.addEventListener("submit", handleSubmit);
```

The thank-you reads the name into `name` **before** `form.reset()` empties the box (Core concept 7). `if (pretendFail)` is short for `if (pretendFail === true)`: an `if` only needs a true or false value, and `pretendFail` already is one.

9. **Test the five paths.** Save and press F5. For each line, write down what you expect, then do it and compare:
   - **Empty.** Press Send with all three fields empty. The sample showed three messages, and focus went to Your name.
   - **Spaces only.** Type three spaces in Your name, `sam@` in Email and `hi` in Message, and press Send. All three messages appeared, and focus went to Your name. Three spaces are not a name, because `.trim()` turned them into `""`.
   - **Bad email.** Change the name to `Sam` and press Send. Name's message went away, and focus went to Email: the left render in the brief.
   - **Short message.** Change the email to `sam@example.com` and press Send. Only the Message line was left, and focus went to Message.
   - **Success.** Change the message to `Hello, I like your site.` and press Send. The button faded, and the status line said **One moment...**. One second later it said **Thanks, Sam. Your message passed every check.**, and all three fields were empty.

   With the Console open the whole time, nothing went red.

10. **See the failed state.** Fill in all three fields correctly again. In the Console, type `pretendFail = true` and press Enter. Then press Send. After one second, the sample showed its failed message in dark red on pale red, with Sam's name, email and message still in the fields. Press **F5** when you are done: the page starts fresh, so `pretendFail` is `false` again.

11. **Modify.** In `findFirstProblem`, around the message check, change `if (firstProblem === null) { firstProblem = messageInput; }` to just `firstProblem = messageInput;`. **Predict:** with an empty name, a good email and `hi` as the message, where will focus go? Write it down, save, press F5 and try it. The sample showed both messages, but focus went to **Message**, the last problem, not the first. The message check ran last and overwrote `firstProblem`. Put the `if` back.

12. **Break it on purpose: reset too early.** In `finishSending`, move `form.reset();` up, so it is the first line of the `else` block, above `const name = …`. Predict what the thank-you will say. Save, press F5, and send a valid form. The sample said **Thanks, . Your message passed every check.** There was no red line: `form.reset()` emptied the name box, and an empty name is a perfectly good string. Move `form.reset();` back to the end of the block.

© 2026 559 Solutions LLC

13. **Debug: a form that ignores you.** In block 1, change `"#visitor-email"` to `"#visitor-mail"`. Save, press F5, and press Send with all three fields empty. Before you read on, write down what you see, and which block you would check first. Then look at the Console.

    In the sample, nothing happened on the page: no messages, no reload. The Console showed:

    ```
    Uncaught TypeError: Cannot read properties of null (reading 'classList')
        at clearError (contact.js:30:9)
        at findFirstProblem (contact.js:37:3)
        at HTMLFormElement.handleSubmit (contact.js:80:24)
    ```

    This message has three lines under it, and you read them from the bottom up. `handleSubmit` ran on the submit, on line 80. It called `findFirstProblem`, which on line 37 called `clearError(emailInput, emailError)`. Inside `clearError`, line 30 read `input.classList`, and `input` was `null`.

    So `emailInput` holds `null`, and a name gets its value in block 1. There, `"#visitor-mail"` matches no element. The page did not reload, because `event.preventDefault()` had already run. Your line numbers match only if your file has the sample's shape. Put the `e` back and test again.

14. **The rating card HTML.** Open `rate.html`. In the `<head>`, under the stylesheet link, add `<script src="rate.js" defer></script>`. Then make three changes to the card:

    ```
    <form class="rating-form" novalidate>
    ```

    ```
    <fieldset class="choices" aria-describedby="rating-error">
    ```

    ```
    <p id="rating-error" class="error"></p>
    ```

    The first two replace the opening tags you have. The error paragraph goes right after `</fieldset>`, before the button. Then, at the end of the card, after `</section>` and before the card's closing `</div>`, add the status line:

    ```
    <p id="rating-status" class="visually-hidden" role="status"></p>
    ```

    Predict: `rate.js` does not exist yet. What will the address end in after Send rating with nothing chosen? And with 4 chosen? Save, open `rate.html`, press F5, and try both. The sample's address ended in `rate.html?` with nothing chosen: the form sent, with no pairs at all, because a group with no choice sends nothing. With 4 chosen, it ended in `rate.html?rating=4`, as in Module 4. Each time, the Console had the red `ERR_FILE_NOT_FOUND` line for `rate.js`. Delete the `?…` part of the address before you go on.

© 2026 559 Solutions LLC

15. **The rating error's spacing.** In `styles.css`, under the `.error:empty` rule, add:

```
.rating-form .error {
  margin: 0 0 16px;
}
```

16. **`rate.js`: read the chosen rating.** Create `rate.js` next to `rate.html`. Type:

```
// rate.js: reads the chosen rating and swaps the form for the thank-you section

// 1. Find elements
const ratingForm = document.querySelector(".rating-form");
const thanks = document.querySelector(".thanks");
const chosenRating = document.querySelector("#chosen-rating");
const firstChoice = document.querySelector("#rating-1");
const ratingError = document.querySelector("#rating-error");
const ratingStatus = document.querySelector("#rating-status");

// 2. Define behavior
// Returns the value of the chosen radio, such as "4", or null if none is
chosen.
function getChosenRating() {
  const chosen = document.querySelector('input[name="rating"]:checked');
  // Continue alone: if chosen is null, return null.
  // Otherwise, return the value of chosen.
}
```

This function only answers a question. It does not touch the page, so you can test it alone. Save, press F5, and type `getChosenRating()` in the Console. The sample gave `null`. Click **4** on the card and type it again. The sample gave `'4'`, with quotes: the value of a radio is text, a string. If you see a red `TypeError` about reading `'value'` instead of `null`, your `if` is missing or comes too late (Problem 4 in Part 4, *Troubleshooting*).

17. **The swap.** Under `getChosenRating`, type:

```
function handleRating(event) {
  event.preventDefault();
  const rating = getChosenRating();
  if (rating === null) {
    ratingError.textContent = "Please choose a number from 1 to 5.";
    firstChoice.focus();
    return;
  }
  // Continue alone: empty the error, put rating in chosenRating,
  // hide the form, show thanks, and put this sentence in
```


© 2026 559 Solutions LLC

```
  // ratingStatus: "Thanks for the feedback. You chose " + rating + " out of 5."
}

// 3. Connect events
ratingForm.addEventListener("submit", handleRating);
```

Read the `if` as a door at the top: the nothing-chosen case leaves right there, with `return`, as in `handleSubmit`. Every line after it can be sure a rating exists. The swap is two `hidden` lines, one `true` and one `false`, because the HTML already holds both states (Module 4).

Save and press F5. Test three ways. **No choice:** click Send rating. The sample showed **Please choose a number from 1 to 5.**, and focus moved to number 1, but with no ring: after a mouse click, Chromium draws the ring only for the keyboard (`:focus-visible`). The keyboard test below shows it.

With 4: click 4, then Send rating. The form was gone, and the card said **You chose 4 out of 5**.

Keyboard only: press F5, put the mouse aside, and press Tab seven times, to Send rating. Press Enter: the message appears, and 1 gets the dark orange ring. Press the right arrow three times, then Tab and Enter. The sample showed the right-hand render in the brief. The hidden button had focus, so focus went back to the page: the next Tab started again at Home, at the top.

18. **Break it on purpose: the whole radio.** In `getChosenRating`, change the last `return` to `return chosen;` (drop `.value`). Predict what the card will say with 4 chosen. Save, press F5, choose 4 and send. The sample said **You chose [object HTMLInputElement] out of 5**, with no red line. `chosen` is the radio element itself. `textContent` turned it into text, and that is the text an element becomes. Put `.value` back.

19. **Write the keyboard test, then run it.** On paper, write one line for each thing a keyboard user must be able to do on these two pages: the key you press, and what should happen. Then put the mouse aside and run every line, on the Contact page and on the Rate page, ticking each one. The sample's plan was nine lines:

    - [ ] F5, then Tab: the five nav links, then Your name, Email, Message and Send message, in that order, each with a ring
    - [ ] On Send message, Enter with all fields empty: three messages, focus in Your name
    - [ ] With a name filled in, in Email with `sam@`, Enter: the form submits, focus stays in Email with its message
    - [ ] In Message, Enter: a new line, no submit
    - [ ] On Send message with a valid form, Enter: **One moment…**, then the thank-you
    - [ ] After the thank-you, Shift+Tab: focus goes back into the form (Core concept 8)
    - [ ] Rate page: six Tabs reach number 1; Tab and Enter with nothing chosen: the message, focus on 1
    - [ ] Arrows choose a number; Tab, Enter: the thank-you section shows the number
    - [ ] After the thank-you, Tab: focus starts again at the top of the page

© 2026 559 Solutions LLC

Keep your plan. Module 11 turns a list like this into a test plan.

20. **Explain it.** Read `contact.js` to yourself, out loud, from top to bottom. For each function, say what causes it to run: a submit, a timer, or another function. Then read `rate.js` the same way, and say why `getChosenRating` returns `null` instead of showing a message itself.

21. **Test the site.** Tick each box in a full-width window and at 390 px in device mode (F12, then Ctrl+Shift+M). Keep the Console open the whole time.

    - [ ] No red line in the Console on any of the five pages, and the gallery and FAQ still work
    - [ ] Empty, spaces-only, bad email and short message each show the right messages, with focus on the first problem
    - [ ] A valid send goes through **sending** to **sent**, names the visitor, and empties the form; `pretendFail = true` shows **failed** with the words kept
    - [ ] In the Elements tab, the form has exactly one of `idle`, `sending`, `sent` or `failed` at a time
    - [ ] The rating card refuses an empty choice with its message, and shows the chosen number after a send
    - [ ] Every line of your keyboard test from step 19 is ticked
    - [ ] Nothing scrolls sideways at 390 px on the Contact page or the Rate page
    - [ ] The changes from steps 11, 12, 13 and 18 are undone, and no message on the page says your message was sent

22. **Save it in Git.** Go to Part 3, *Finishing work in Git*, and publish both features with the save routine.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

The Contact page and the rating card work on your computer. Now you put them online with the same save routine as in Modules 1 to 9: `git status`, `git add .`, `git commit`, `git push`, then check the live site. Two things are new this time. You search your files for the test changes you made on purpose, so none of them is published. And you run your keyboard test from Project step 19 again, on the live pages.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 to 9.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **Search for leftover test changes (NEW).** This project asked you to break things on purpose four times. Make sure every change is undone before Git saves it. In VS Code, open **Edit → Find in Files** (**Ctrl+Shift+F**) and search for `visitor-mail`, then for `return chosen;`. Each search should find nothing. Then open `finishSending` and check that `form.reset();` is the last line of the `else` block, and open `findFirstProblem` and check that every `firstProblem = …` after the name check sits inside `if (firstProblem === null)`.

3. **Read the Console before you commit.** Open each of your five pages in Edge or Chrome with the Console open (**F12**, then the **Console** tab), and press **F5** on each. There should be no red line on any of them. On the Contact page and the Rate page, press the send button once with everything empty: an error in a submit handler only shows up after a submit. Commit only pages whose test from Project step 21 you have just run and seen pass.

4. **See what changed.** Type:

   ```
   git status
   ```

   Under **Changes not staged for commit** you should see `modified:` for `contact.html`, `rate.html` and `styles.css`. Under **Untracked files** you should see `contact.js` and `rate.js`. If another page or script shows up, you changed a file this project does not touch. Open it and check what changed before you go on.

5. **Stage the changes.** Type:

   ```
   git add .
   ```

   It prints nothing when it works. (A warning that mentions LF and CRLF is harmless.) Run `git status` again. Under **Changes to be committed** you should now see five lines: `modified:` for the two pages and `styles.css`, and `new file:` for the two scripts.

© 2026 559 Solutions LLC

6. **Commit.** Type:

   ```
   git commit -m "Add form checks to Contact and JavaScript to the rating card"
   ```

   On the sample site, Git printed `5 files changed, 199 insertions(+), 6 deletions(-)` on the second line, then `create mode 100644 contact.js` and `create mode 100644 rate.js`. The six deletions are the old versions of lines you changed: the two `<form>` tags, the three fields and the `<fieldset>` tag. Your commit id and line counts will differ.

7. **Push.** Type:

   ```
   git push
   ```

   The last line ends in `main -> main`.

8. **Check that Git is done.** Type `git status`. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. Your new commit is on top, with your Module 9 commit (or your latest one) under it.

9. **Check the live pages with the Console open.** Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), then open `https://your-username.github.io/my-site/contact.html` with the Console open, and press F5. If the page acts like the old version, press **Ctrl+Shift+R** to reload without the saved copy, as in Module 9. Then run the five paths from Project step 9. The live page sends nothing, exactly like the copy on your computer: the note on the page says so.

10. **Run your keyboard test on the live site (NEW).** Put the mouse aside and run your plan from Project step 19 again, line by line, on the live Contact and Rate pages. Tick each line again. A test you wrote down can be run the same way every time, by you or by someone else. That is the idea Module 11 builds on.

11. **Test on a phone.** If you have a phone, open your live Contact page on it. Press Send with the fields empty and check that you can read every message. Then rate the lesson on the live Rate page. Nothing should scroll sideways.

The steps in this packet were tested with files opened from disk in Chromium, not on a live site.

> **Checkpoint.** `git status` says `nothing to commit, working tree clean`. `git log --oneline` shows your new commit on top. On the live site, the Console shows no red line on the Contact or Rate page, every line of your keyboard test is ticked, and no message on the Contact page says your message was sent.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: six common problems

Each problem has the symptom you see, why it happens, and the fix. Each one was made on purpose in the sample site and run in Chromium 141, with the files opened from disk as in the steps. The Console messages are quoted as Chromium printed them. The line numbers are the sample's: yours match only if your file has the same shape. After each fix, save, press **F5**, and run the test from Project step 21 again.

1. The page reloads, and the address gains `?visitor-name=…`

Symptom. You press Send message, the page flashes, and the form comes back empty. The address now ends in something like `contact.html?visitor-name=Sam&visitor-email=&message=`. Any message you saw is gone.

Why. The browser did its default: it sent the form and loaded the page again. With `novalidate` on the form, the browser does not check anything itself, so an empty form goes straight through. Your handler either never ran, or did not stop the default. The sample got this three ways:

What was wrong	What the Console showed
A misspelled file name, `<script src="contacts.js" defer>`	`Failed to load resource: net::ERR_FILE_NOT_FOUND`, with `contacts.js` beside it
`defer` missing from the `<script>` line	`Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')` at `contact.js:94:6`, the line in block 3
`event.preventDefault();` commented out or missing	nothing at all; the handler writes its messages, then the reload wipes them

Fix. Read the Console first. A red line names the file or the line to fix: compare the file name with the Explorer, or add `defer` (Module 9). No red line means the handler ran: check that `event.preventDefault();` is its first line. Then delete the `?…` part of the address and press Enter, so you test a clean page.

2. Nothing happens, and the Console shows a TypeError about `null`

Symptom. You press Send message. No message appears, and the page does not reload. The Console shows a red message with three lines under it, such as:

```
Uncaught TypeError: Cannot set properties of null (setting 'textContent')
    at clearError (contact.js:29:28)
    at findFirstProblem (contact.js:36:3)
    at HTMLFormElement.handleSubmit (contact.js:80:24)
```


© 2026 559 Solutions LLC

Why. A name in block 1 holds `null`, because its selector matches no element. The script found out only when a function used that name. Here the selector was `"#nameerror"` for `id="name-error"`, so `nameError` was `null`, and `clearError` tried to set its `textContent`. With `"#visitor-mail"`, the message was `Cannot read properties of null (reading 'classList')` at `contact.js:30:9` instead (Project step 13). The page did not reload, because `event.preventDefault()` ran before the error.

Fix. Read the lines under the message from the bottom up: which function called which, and with what. The top line is where `null` was used. Find which name was passed in, then compare its selector in block 1 with the id in `contact.html`, character by character.

3. The status line has no color

Symptom. The status line says **One moment...** and then the thank-you, but in plain dark text on the page background, with no pale green or pale red box.

Why. Each state rule, such as `.sent + #contact-status`, matches only when the status paragraph comes **right after** the form, with nothing between them. In the sample, with `<p id="contact-status">` moved inside the form, the text still changed, but its background stayed transparent. The padding from `#contact-status` still applied, so the text sat a little indented. The same happens if another element sits between `</form>` and the status line.

Fix. Put `<p id="contact-status" role="status"></p>` directly after `</form>`, as in the brief. Then check in the Elements tab (F12) that it is the form's next sibling.

4. TypeError: Cannot read properties of null (reading 'value')

Symptom. On the Rate page, Send rating with nothing chosen does nothing at all: no message, no ring on 1. The Console shows:

```
Uncaught TypeError: Cannot read properties of null (reading 'value')
    at getChosenRating (rate.js:15:17)
    at HTMLFormElement.handleRating (rate.js:20:18)
```

With a number chosen, the card works.

Why. With nothing chosen, `querySelector('input[name="rating"]:checked')` gives `null` (Core concept 2). The sample's `getChosenRating` had no `if`, so it went straight to `return chosen.value;`, and `null` has no properties. The same message comes if the `if` is there but checks something else, such as `chosen === undefined`.

Fix. Check for `null` first, with `===`, and return `null` from inside that `if`. Only the line after it reads `.value`, when `chosen` is sure to be a radio.

© 2026 559 Solutions LLC

5. The thank-you section shows when the page loads

Symptom. The Rate page opens with the form **and** the thank-you section, and the span is empty: **You chose out of 5**. After a send, the form disappears and the number fills in.

Why. A `display` rule on the section beats the `hidden` attribute (Module 9, Core concept 8). In the sample, with `display: block;` added to `.thanks`, the section's `hidden` was `true` at load, but its display was `block`, so it showed anyway.

Fix. Give `.thanks` no `display` rule, as in the Module 4 brief. The same goes for `.rating-form`: a `display` rule there would keep the form visible after the swap.

6. The form accepts `sam@example`

Symptom. You type `sam@example`, with no dot, and the email passes. `sam@` is refused, but `sam@example` is not.

Why. Your script asks the browser, with `emailInput.checkValidity()`, and the browser's `type="email"` rule needs only something before the `@` and something after it. In the sample, `checkValidity()` gave `true` for `sam@example` and `sam@example.com`, and `false` for `sam@`, `@example.com`, `sam@@example.com` and `sam example@x.com`.

Fix. Nothing is broken. The check in the browser only catches typing mistakes; no rule in the browser can prove an address is real. The way to know is to send a message to it, and that happens on a server, after the form. Leave the rule as it is.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 11. If a line is a 1 or a 2, reread that Core concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can catch a form with a submit listener, stop the reload with event.preventDefault(), and name two ways a visitor fires submit	☐	☐	☐	☐
I can read a field with .value, trim it, and refuse a value that is empty or only spaces	☐	☐	☐	☐
I can read the chosen radio with :checked, and check for null with === before I read its value	☐	☐	☐	☐
I can say what required, type="email", minlength and pattern check, and why minlength needs required	☐	☐	☐	☐
I can use novalidate with my own checks, and explain why the HTML rules stay in place	☐	☐	☐	☐
I can tie an error message to its field with aria-describedby, and say why that is the only ARIA the form needs	☐	☐	☐	☐
I can find the first field with a problem, move focus to it, and write messages that say what to do	☐	☐	☐	☐
I can confirm success in a role="status" element, read what I need before .reset(), and show four states as classes	☐	☐	☐	☐
I can explain why client-side validation is user experience, not security, and what a server must still do	☐	☐	☐	☐

Three questions to answer in writing

1. In Project step 12, moving `form.reset()` up gave **Thanks, . Your message passed every check.** with no red line. In Project step 13, one wrong selector gave a red `TypeError` and no messages at all. Explain why one mistake gives an error and the other does not, and what the clue was in each case.

© 2026 559 Solutions LLC

2. Pick one prediction from the Exercise or the project where your first answer was wrong. What did you write down, what did the page or the Console show, and what do you understand now that you did not before?

3. A friend wrote this `handleRating`. What does the card do when the visitor presses Send rating with nothing chosen? What does it do with 3 chosen? And what would you tell them to change?

```
function handleRating(event) {
  event.preventDefault();
  const rating = getChosenRating();
  if (rating === "") {
    ratingError.textContent = "Please choose a number from 1 to 5.";
    firstChoice.focus();
    return;
  }
  ratingError.textContent = "";
  chosenRating.textContent = rating;
  ratingForm.hidden = true;
  thanks.hidden = false;
}
```


© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **Errors that flash and vanish.** `event.preventDefault()` is missing, or it is not the handler's first line, or it never runs. Your handler wrote the messages, then the browser did its default: it sent the form, which adds the `?name=value` pairs to the address, and loaded the page again. The new page starts fresh, so the messages are gone. Make `event.preventDefault();` the first line of the submit handler. (Core concept 1; Exercise step 9)
2. **What value does this variable hold?** `nameInput.value` holds every character the visitor typed: two spaces, `Ana`, one space. That is the string `" Ana "`, six characters long. `nameInput.value.trim()` gives a new string without the spaces at both ends: `"Ana"`, three characters. The box itself still holds the six; `.trim()` does not change the original. (Core concept 2)
3. **Why keep `required` and `minlength`.** `novalidate` only stops the browser's message bubbles and its block on the submit. The attributes still describe the rules, and the browser still uses them: `checkValidity()` and `validationMessage` read them, and the Exercise's email check is `emailInput.checkValidity()`, which uses `type="email"`. They also tell assistive technology about the field: in Chromium's accessibility tree, the project's Name box was a required textbox. And anyone who reads your HTML sees the rules at a glance. (Core concepts 3 and 4)

Common misconception

"If my JavaScript checks every field, the data that reaches me will always be clean." Your checks run in the visitor's browser, and the visitor controls that browser. One line in the Console removed every `required` attribute in Core concept 9. A visitor can also switch JavaScript off, change your script in DevTools, or skip your page and send data straight to the server.

So your checks help honest visitors fix mistakes, and stop nobody who wants to get around them. Client-side validation is user experience, not security. The server must check every value again, every time, before it trusts it. (Core concept 9)

Exercise checks

These are the results with the lesson's code, checked in Chromium.

- **Step 4.** Three messages: **Please enter your name.**, **Please enter your email address.** and **Please write at least 10 characters.** Focus goes to Name, the first field with a problem. The form keeps the class `idle`. (Core concepts 5 and 6)

© 2026 559 Solutions LLC

- **Step 5.** Three messages again. Name says **Please enter your name.**, because `.trim()` turned the three spaces into `""`. Email says **That email address looks incomplete. Check the @ and the part after it.** Message says **Please write at least 10 characters.** Focus goes to Name. (Core concepts 2 and 4)
- **Step 6.** First the `sending` state: the button fades and is disabled, and the status says **Sending...**. One second later, the `sent` state: **Thanks, Your Name. Your message was sent.** in green, and all three fields are empty. (Core concepts 7 and 8)
- **Step 7.** After one second, the `failed` state: **Sorry, your message did not go through. Please try again.** in red, and your values are still in the fields. (Core concept 8)
- **Step 8.** Enter in the Name or Email box submits the form, and focus lands on the first field with a mistake. With a name and `you@`, focus went to Email. In the Message box, Enter starts a new line, so you Tab to Send and press Enter there. (Core concepts 1 and 6)
- **Step 9.** The page reloads, the address ends in `index.html?name=Your+Name&email=&message=`, and the form is empty with no messages. The Console shows nothing red. (Core concept 1)
- **Step 10.** Three spaces pass the check, and the form "sends." The thank-you trims the name, so it says **Thanks, . Your message was sent.** (Core concept 2)

Self-assessment questions

1. **Why one mistake gives an error and the other does not.** In step 12, every line is valid. `form.reset()` empties the Name box, and `nameInput.value.trim()` then gives `""`, an empty string. Joining `""` into a sentence is allowed, so nothing fails and nothing is printed. The clue is the page itself: the comma right after **Thanks,** with no name.

   In step 13, `querySelector("#visitor-mail")` gave `null`, and `clearError` read `input.classList` with `input` holding `null`. `null` has no properties at all, so the browser stopped the handler with a `TypeError`. The clue is the message: its three lines lead from `handleSubmit` to `clearError`, and back to the selector in block 1.

2. **A prediction you got wrong.** There is no single answer. A good answer quotes what you wrote down before you saved, says what the page or the Console really showed, and names the idea you understand now. For example: that `novalidate` with no script lets an empty form through (Project step 3), or that a group with no choice sends no pair at all (Project step 14).

3. **The friend's `handleRating`.** With nothing chosen, `getChosenRating()` gives `null`, not `""`. `null === ""` is `false`, so the `if` is skipped, and there is no red line. The form disappears and the thank-you shows **You chose out of 5**, with nothing in the span: setting `textContent` to `null` empties it. With 3 chosen, the card works and shows **You chose 3 out of 5**.

   Tell them to compare with what the function really returns: `if (rating === null)`. Their version also never fills `ratingStatus`, so a screen reader hears nothing after the swap. They should add that line from Project step 17. (Core concept 2; Project steps 16 and 17)

© 2026 559 Solutions LLC

Project checks

- **Step 3.** The page reloads with the address ending in `contact.html?visitor-name=&visitor-email=&message=`, and the Console has one red line, `Failed to load resource: net::ERR_FILE_NOT_FOUND`, for `contact.js`.
- **Step 3, continue alone.** The email and message fields of the sample form:

```
  <label for="visitor-email">Email</label>
  <input type="email" id="visitor-email" name="visitor-email" required aria-
describedby="email-error">
  <p id="email-error" class="error"></p>

  <label for="message">Message</label>
  <textarea id="message" name="message" rows="5" required minlength="10" aria-
describedby="message-error"></textarea>
  <p id="message-error" class="error"></p>
```

- **Step 4, continue alone.**

```
.sent + #contact-status {
  color: #1e5b2a;
  background-color: #e3f4e5;
}

.failed + #contact-status {
  color: #8c1d18;
  background-color: #fbe4e2;
}
```

- **Step 5, continue alone.** The four lines are in the whole `contact.js` at the end of this section. `emailError` gave `<p id="email-error" class="error"></p>` and `pretendFail` gave `false`.
- **Step 6, continue alone.**

```
function showError(input, errorElement, text) {
  errorElement.textContent = text;
  input.classList.add("invalid");
}

function clearError(input, errorElement) {
  errorElement.textContent = "";
  input.classList.remove("invalid");
}
```


© 2026 559 Solutions LLC

`showError(nameInput, nameError, "Test")` showed **Test** in bold red and a red border; `clearError` removed both; `form.className` gave `'contact-form sent'`.

- **Step 7, continue alone.** The sample's email and message checks, with its own messages:

```
  const email = emailInput.value.trim();
  if (email === "") {
    showError(emailInput, emailError, "Please add your email, so I can write
back.");
    if (firstProblem === null) { firstProblem = emailInput; }
  } else if (!emailInput.checkValidity()) {
    showError(emailInput, emailError, "That email looks incomplete. Check the @
and the part after it.");
    if (firstProblem === null) { firstProblem = emailInput; }
  }

  const message = messageInput.value.trim();
  if (message.length < 10) {
    showError(messageInput, messageError, "Tell me a little more: at least 10
characters.");
    if (firstProblem === null) { firstProblem = messageInput; }
  }
```

  With empty fields, `findFirstProblem()` returned the Name box; with all three filled in correctly, it returned `null`.

- **Step 8, continue alone.** The failed branch:

```
    setState("failed");
    statusMessage.textContent = "Sorry, that did not work. Your message is still
here, so please try again.";
```

- **Steps 9 and 10.** As described in the steps. In the failed state, the form's class is `contact-form failed` and the three values stay.
- **Step 11.** Both messages show, and focus goes to Message instead of Your name.
- **Step 12. Thanks, . Your message passed every check.**, with no red line.
- **Step 13.** No messages and no reload. The Console shows `Uncaught TypeError: Cannot read properties of null (reading 'classList')`, then `at clearError (contact.js:30:9)`, `at findFirstProblem (contact.js:37:3)` and `at HTMLFormElement.handleSubmit (contact.js:80:24)`.
- **Step 14.** `rate.html?` with nothing chosen, `rate.html?rating=4` with 4 chosen, and the red `ERR_FILE_NOT_FOUND` line for `rate.js` each time.

© 2026 559 Solutions LLC

- **Step 16, continue alone.**

  ```
    if (chosen === null) {
      return null;
    }
    return chosen.value;
  ```

  `getChosenRating()` gave `null`, then `'4'` after a click on 4.

- **Step 17, continue alone.** The swap:

  ```
    ratingError.textContent = "";
    chosenRating.textContent = rating;
    ratingForm.hidden = true;
    thanks.hidden = false;
    ratingStatus.textContent = "Thanks for the feedback. You chose " + rating + "
  out of 5.";
  ```

  The first line clears the message left from an earlier try with nothing chosen.

- **Step 18. You chose [object HTMLInputElement] out of 5**, with no red line.

- **Step 21.** If you followed the brief, every box is ticked. The most common misses: a status line that is not right after the form (Problem 3 in Part 4), and a test change from steps 11 to 13 or 18 that was never undone.

Here are the sample's finished files. First, the whole `contact.js`, 94 lines:

```
// contact.js: checks the Contact form, shows clear messages, and shows four states

// 1. Find elements
const form = document.querySelector(".contact-form");
const nameInput = document.querySelector("#visitor-name");
const emailInput = document.querySelector("#visitor-email");
const messageInput = document.querySelector("#message");
const nameError = document.querySelector("#name-error");
const emailError = document.querySelector("#email-error");
const messageError = document.querySelector("#message-error");
const sendButton = document.querySelector(".contact-form button");
const statusMessage = document.querySelector("#contact-status");

// For testing only: type pretendFail = true in the Console to see the failed state.
let pretendFail = false;

// 2. Define behavior
function setState(stateName) {
```


© 2026 559 Solutions LLC

```
  form.classList.remove("idle", "sending", "sent", "failed");
  form.classList.add(stateName);
}

function showError(input, errorElement, text) {
  errorElement.textContent = text;
  input.classList.add("invalid");
}

function clearError(input, errorElement) {
  errorElement.textContent = "";
  input.classList.remove("invalid");
}

// Checks every field. Returns the first field with a problem, or null.
function findFirstProblem() {
  let firstProblem = null;
  clearError(nameInput, nameError);
  clearError(emailInput, emailError);
  clearError(messageInput, messageError);

  const name = nameInput.value.trim();
  if (name === "") {
    showError(nameInput, nameError, "I'd love to know who is writing. Please add your
name.");
    firstProblem = nameInput;
  }

  const email = emailInput.value.trim();
  if (email === "") {
    showError(emailInput, emailError, "Please add your email, so I can write back.");
    if (firstProblem === null) { firstProblem = emailInput; }
  } else if (!emailInput.checkValidity()) {
    showError(emailInput, emailError, "That email looks incomplete. Check the @ and
the part after it.");
    if (firstProblem === null) { firstProblem = emailInput; }
  }

  const message = messageInput.value.trim();
  if (message.length < 10) {
    showError(messageInput, messageError, "Tell me a little more: at least 10
characters.");
    if (firstProblem === null) { firstProblem = messageInput; }
  }

  return firstProblem;
}
```


© 2026 559 Solutions LLC

```
function finishSending() {
  if (pretendFail) {
    setState("failed");
    statusMessage.textContent = "Sorry, that did not work. Your message is still here,
so please try again.";
  } else {
    const name = nameInput.value.trim();
    setState("sent");
    statusMessage.textContent = "Thanks, " + name + ". Your message passed every
check.";
    form.reset();
  }
  sendButton.disabled = false;
}

function handleSubmit(event) {
  event.preventDefault();
  statusMessage.textContent = "";
  const firstProblem = findFirstProblem();
  if (firstProblem !== null) {
    setState("idle");
    firstProblem.focus();
    return;
  }
  setState("sending");
  statusMessage.textContent = "One moment...";
  sendButton.disabled = true;
  // Stand-in for a real server: wait one second, then finish.
  setTimeout(finishSending, 1000);
}

// 3. Connect events
form.addEventListener("submit", handleSubmit);
```

The whole `rate.js`, 37 lines:

```
// rate.js: reads the chosen rating and swaps the form for the thank-you section

// 1. Find elements
const ratingForm = document.querySelector(".rating-form");
const thanks = document.querySelector(".thanks");
const chosenRating = document.querySelector("#chosen-rating");
const firstChoice = document.querySelector("#rating-1");
const ratingError = document.querySelector("#rating-error");
const ratingStatus = document.querySelector("#rating-status");
```


© 2026 559 Solutions LLC

```
// 2. Define behavior
// Returns the value of the chosen radio, such as "4", or null if none is chosen.
function getChosenRating() {
  const chosen = document.querySelector('input[name="rating"]:checked');
  if (chosen === null) {
    return null;
  }
  return chosen.value;
}

function handleRating(event) {
  event.preventDefault();
  const rating = getChosenRating();
  if (rating === null) {
    ratingError.textContent = "Please choose a number from 1 to 5.";
    firstChoice.focus();
    return;
  }
  ratingError.textContent = "";
  chosenRating.textContent = rating;
  ratingForm.hidden = true;
  thanks.hidden = false;
  ratingStatus.textContent = "Thanks for the feedback. You chose " + rating + " out of
5.";
}

// 3. Connect events
ratingForm.addEventListener("submit", handleRating);
```

In `styles.css`, the only changes are the `--error` line in `:root` and the new group above the wider-screens block: the rules from Project steps 4 and 15, with the "Continue alone" part above filled in. In `contact.html` and `rate.html`, the changes are the `<script>` line in the `<head>` and the new and changed lines from Project steps 3 and 14.

NEXT

Module 11: Debugging and Testing.
Free lesson: 559solutions.com/code559/module-11-debugging-testing/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC